Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Theory, Edge Cases & Code Examples

- ✓ Overview
- ✓ Understanding Race Conditions
- ✓ The Four Coffman Conditions for Deadlock
- ✓ Deadlock Strategies
- ✓ Mission-Critical Systems Impact

→ Swipe to learn →

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Theory

📌 1. Race Conditions and Deadlocks - Overview

Definitions:

Concurrency Bug	Definition	Symptom
Race Condition	Multiple threads access shared mutable state; at least one writes; outcome depends on unpredictable timing	Non-deterministic results, data corruption
Deadlock	Two or more threads permanently blocked, each waiting for resources held by others	System hang, no progress

Key Characteristics:
- Both manifest as **Heisenbugs** - bugs that disappear when you try to observe them (debugging changes timing)
- Race conditions cause incorrect results
- Deadlocks cause complete program freeze

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Theory

📌 2. Understanding Race Conditions (1/2)

At the Machine Level:

Seemingly atomic operations decompose into multiple instructions:

```
++counter;  // Looks atomic, but...

// Actually executed as:
// 1. LOAD counter into register (R1 = memory[counter])
// 2. INCREMENT register (R1 = R1 + 1)
// 3. STORE register to memory (memory[counter] = R1)
```

Race Scenario:

Time	Thread 1	Thread 2	Counter Value
T0	-	-	100
T1	LOAD (R1=100)	-	100
T2	-	LOAD (R2=100)	100
T3	INCREMENT (R1=101)	-	100
T4	-	INCREMENT (R2=101)	100
T5	STORE (101)	-	101
T6	-	STORE (101)	101

Expected: 102, Actual: 101 - Lost one increment

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Theory

📌 2. Understanding Race Conditions (2/2)

Common Race Patterns:

Pattern	Description	Example
Lost Update	Concurrent read-modify-write loses changes	Two threads increment same counter
Read-Modify-Write Race	Multi-step update without atomicity	`counter++`, `balance -= amount`
Check-Then-Act (TOCTOU)	Condition changes between check and action	`if (cache.empty()) cache.insert()`
Publication Race	Object published before full initialization	Publishing pointer before constructor completes

---

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Theory

📌 3. The Four Coffman Conditions for Deadlock

All four must hold simultaneously for deadlock to occur:

#	Condition	Meaning	Example
1	**Mutual Exclusion**	Resources cannot be shared (only one owner)	Only one thread can hold a mutex
2	**Hold and Wait**	Hold resources while waiting for others	Holding lock A while waiting for lock B
3	**No Preemption**	Resources cannot be forcibly taken	Can't force a thread to release a lock
4	**Circular Wait**	Circular chain of waiting threads	T1→T2→T3→T1 (where → means "waits for")

Breaking Any One Condition Prevents Deadlock:

Break Which?	How?	Example Technique
Circular Wait	Enforce global lock ordering	Always lock in address order
Hold and Wait	Acquire all locks atomically	`std::lock(m1, m2, m3)`
No Preemption	Allow timeout-based release	`try_lock_for()` with retry
Mutual Exclusion	Use shareable resources	`shared_mutex` for readers

ABBA Deadlock Example:

```
// Thread 1:          Thread 2:
lock(A)              lock(B)
lock(B)  ← waits     lock(A)  ← waits
         DEADLOCK!
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Theory

📌 4. Deadlock Strategies

Three Approaches:

Strategy	Description	Pros	Cons
Prevention	Design system to never deadlock (break Coffman condition)	No runtime cost, guaranteed safe	Requires design discipline
Avoidance	Dynamically ensure safe states (Banker's algorithm)	Theoretical elegance	Complex, impractical for general use
Detection + Recovery	Allow deadlocks, detect (timeouts/graphs), recover (abort/kill)	Flexible	High complexity, performance cost

Most Common: Prevention via Lock Ordering

```
// ✅ Correct: Same order everywhere
Thread 1: lock(A); lock(B);
Thread 2: lock(A); lock(B);  // No deadlock

// ❌ Wrong: Different orders
Thread 1: lock(A); lock(B);
Thread 2: lock(B); lock(A);  // Deadlock!
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Theory

📌 5. Mission-Critical Systems Impact

Autonomous Driving Example:

Component	Race Condition Impact	Deadlock Impact
Sensor Fusion (30Hz)	Inconsistent world model → wrong obstacle positions	Thread freeze → vehicle uses stale data at 120 km/h
Path Planning (10Hz)	Erratic path decisions → sudden lane changes	No new paths generated → follows obsolete trajectory
Control Loop (100Hz)	Inconsistent brake/throttle commands → jerky motion	Complete freeze → loss of vehicle control

Why Prevention is Mandatory:

- **Heisenbugs**: Intermittent failures impossible to reproduce in testing
- **Real-time deadlines**: No time for detection/recovery mechanisms
- **Safety certification (ISO 26262)**: Requires demonstrable absence of data races
- **Production debugging impractical**: Can't add logging when car is at highway speed

Required Practices:

- Static analysis during development (ThreadSanitizer)
- Formal verification of lock orderings
- Lock-free algorithms where feasible
- Systematic code reviews for shared data access

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Theory

📌 6. Key Debugging Tools

Detection Methods:

Tool	Type	What It Detects	Overhead	Use Case
ThreadSanitizer (TSan)	Dynamic analysis	Data races via happens-before tracking	5-15x slowdown	Development, CI testing
Helgrind (Valgrind)	Dynamic analysis	Data races via lock-set algorithm	20-50x slowdown	Legacy code without recompilation
Timeout Detection	Runtime check	Potential deadlocks (lock wait > threshold)	Low	Production monitoring
Wait-for Graph	Runtime analysis	Circular dependencies	Medium	Database systems

Best Practice:
- Use TSan during development and in CI pipeline
- Prevention through design is far superior to runtime detection

---

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Edge Cases

📌 1. Read-Modify-Write Race on "Thread-Safe" Containers

```
std::mutex mtx;
std::unordered_map<int, int> cache;

void increment_cached_value(int key) {
    std::lock_guard<std::mutex> lock(mtx);
    int current = cache[key];  // Read
    // ❌ Lock released here when lock destructs

    // Some computation
    int updated = current + 1;  // Modify

    std::lock_guard<std::mutex> lock2(mtx);
    cache[key] = updated;  // Write
    // ❌ Race! Another thread might have updated cache[key] between locks
}
```

This appears safe (operations are locked) but creates a race because the lock is released between read and write. Another thread can interleave its update, causing lost updates.

Fix: Extend critical section

```
void increment_cached_value_safe(int key) {
    std::lock_guard<std::mutex> lock(mtx);
    cache[key]++;  // ✅ Atomic read-modify-write under single lock
}
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Edge Cases

📌 2. ABBA Deadlock with Pointer-Based Locking

```
class Account {
public:
    std::mutex mtx;
    int balance;
};

void transfer(Account* from, Account* to, int amount) {
    std::lock_guard<std::mutex> lock1(from->mtx);  // ❌ Order depends on arguments!
    std::lock_guard<std::mutex> lock2(to->mtx);

    from->balance -= amount;
    to->balance += amount;
}

// Thread 1: transfer(&accountA, &accountB, 100)  → locks A, then B
// Thread 2: transfer(&accountB, &accountA, 50)   → locks B, then A  → DEADLOCK
```

The lock order depends on function arguments, creating unpredictable lock ordering across different call sites.

Fix: Lock by address order

```
void transfer_safe(Account* from, Account* to, int amount) {
    Account* first = (from < to) ? from : to;  // ✅ Always lock lower address first
    Account* second = (from < to) ? to : from;

    std::lock_guard<std::mutex> lock1(first->mtx);
    std::lock_guard<std::mutex> lock2(second->mtx);

    from->balance -= amount;
    to->balance += amount;
}
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Edge Cases

📌 3. Time-of-Check to Time-of-Use (TOCTOU) Race

```
std::mutex mtx;
std::unordered_map<std::string, User*> users;

User* get_user(const std::string& name) {
    std::lock_guard<std::mutex> lock(mtx);
    if (users.find(name) != users.end()) {  // ❌ Check
        return users[name];
    }
    return nullptr;
}

void update_user(const std::string& name) {
    User* user = get_user(name);  // Lock released after return

    if (user != nullptr) {  // ❌ Use - but user might be deleted by another thread!
        user->update();  // Potential use-after-free
    }
}
```

The pointer becomes dangling if another thread removes the user between `get_user()` returning and `user->update()`.

Fix: Hold lock across check and use

```
void update_user_safe(const std::string& name) {
    std::lock_guard<std::mutex> lock(mtx);  // ✅ Hold lock for both check and use
    auto it = users.find(name);
    if (it != users.end()) {
        it->second->update();
    }
}
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Edge Cases

📌 4. Publish-Subscribe Race with Incomplete Initialization (1/2)

```
class Config {
public:
    std::vector<int> settings;
    bool ready = false;
};

Config* global_config = nullptr;

void publisher_thread() {
    Config* cfg = new Config();
    cfg->settings = {1, 2, 3, 4, 5};  // Initialize

    global_config = cfg;  // ❌ Publish before ready flag set
    cfg->ready = true;     // Other threads might read settings before this
}

void subscriber_thread() {
    while (global_config == nullptr) {  // Wait for publication
        std::this_thread::yield();
    }

    // ❌ global_config->ready might still be false
    // ❌ settings might not be fully initialized (memory reordering)
    if (global_config->ready) {
        process(global_config->settings);
    }
}
```

Without proper memory ordering, the subscriber might see `global_config` as non-null but with incomplete initialization.

📌 4. Publish-Subscribe Race with Incomplete Initialization (2/2)

Fix: Use memory barriers

```
std::atomic<Config*> global_config{nullptr};

void publisher_thread_safe() {
    Config* cfg = new Config();
    cfg->settings = {1, 2, 3, 4, 5};
    cfg->ready = true;

    global_config.store(cfg, std::memory_order_release);  // ✅ Release barrier
}

void subscriber_thread_safe() {
    Config* cfg = global_config.load(std::memory_order_acquire);  // ✅ Acquire barrier

    while (cfg == nullptr) {
        std::this_thread::yield();
        cfg = global_config.load(std::memory_order_acquire);
    }

    process(cfg->settings);  // Guaranteed fully initialized
}
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Edge Cases

📌 5. Livelock - Threads Actively Changing State Without Progress (1/2)

```
std::atomic<bool> flag1{false};
std::atomic<bool> flag2{false};

void polite_thread1() {
    while (true) {
        flag1 = true;

        if (flag2) {  // If other thread wants access
            flag1 = false;  // ❌ Be polite, back off
            std::this_thread::yield();
            continue;  // Try again
        }

        // Critical section
        break;
    }

    flag1 = false;
}

void polite_thread2() {
    while (true) {
        flag2 = true;

        if (flag1) {
            flag2 = false;  // ❌ Be polite, back off
            std::this_thread::yield();
            continue;
        }

        // Critical section
        break;
    }

    flag2 = false;
}
```

Both threads continuously back off for each other, making progress but never entering the critical section—a livelock.

📌 5. Livelock - Threads Actively Changing State Without Progress (2/2)

Fix: Use proper synchronization

```
std::mutex mtx;

void proper_thread() {
    std::lock_guard<std::mutex> lock(mtx);  // ✅ Guaranteed progress
    // Critical section
}
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Code Examples

📌 1. Classic Data Race - Lost Updates

Multiple threads perform non-atomic read-modify-write, causing lost updates. The final value is unpredictable and less than expected.

```
#include <iostream>
#include <thread>
#include <vector>

int shared_counter = 0;  // ❌ No synchronization

void increment_race() {
    for (int i = 0; i < 100000; ++i) {
        ++shared_counter;  // Read-modify-write race
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 10; ++i) {
        threads.emplace_back(increment_race);
    }

    for (auto& t : threads) {
        t.join();
    }

    std::cout << "Expected: 1000000, Actual: " << shared_counter << "\n";
    // Typical output: 300000-800000 (lost updates)
}
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Code Examples

📌 2. Fixing Race with Mutex

Protecting the critical section with a mutex ensures atomicity of the read-modify-write operation.

```
#include <iostream>
#include <thread>
#include <vector>
#include <mutex>

std::mutex mtx;
int shared_counter = 0;

void increment_safe() {
    for (int i = 0; i < 100000; ++i) {
        std::lock_guard<std::mutex> lock(mtx);
        ++shared_counter;  // ✅ Protected
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 10; ++i) {
        threads.emplace_back(increment_safe);
    }

    for (auto& t : threads) {
        t.join();
    }

    std::cout << "Counter: " << shared_counter << "\n";  // Always 1000000
}
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Code Examples

📌 3. ABBA Deadlock Scenario

Thread 1 holds A and waits for B; Thread 2 holds B and waits for A. Classic circular wait deadlock.

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex mutex_A;
std::mutex mutex_B;

void thread_function_1() {
    std::lock_guard<std::mutex> lock_A(mutex_A);  // Lock A first
    std::this_thread::sleep_for(std::chrono::milliseconds(10));

    std::cout << "Thread 1 trying to lock B...\n";
    std::lock_guard<std::mutex> lock_B(mutex_B);  // Wait for B

    std::cout << "Thread 1 got both locks\n";
}

void thread_function_2() {
    std::lock_guard<std::mutex> lock_B(mutex_B);  // ❌ Lock B first (opposite order)
    std::this_thread::sleep_for(std::chrono::milliseconds(10));

    std::cout << "Thread 2 trying to lock A...\n";
    std::lock_guard<std::mutex> lock_A(mutex_A);  // Wait for A

    std::cout << "Thread 2 got both locks\n";
}

int main() {
    std::thread t1(thread_function_1);
    std::thread t2(thread_function_2);

    t1.join();  // ❌ Hangs forever - deadlock
    t2.join();
}
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Code Examples

📌 4. Fixing Deadlock with Lock Ordering

Both threads acquire locks in the same order (A then B), preventing circular wait.

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex mutex_A;
std::mutex mutex_B;

void safe_thread_1() {
    std::lock_guard<std::mutex> lock_A(mutex_A);  // ✅ Same order
    std::lock_guard<std::mutex> lock_B(mutex_B);
    std::cout << "Thread 1 got both locks\n";
}

void safe_thread_2() {
    std::lock_guard<std::mutex> lock_A(mutex_A);  // ✅ Same order
    std::lock_guard<std::mutex> lock_B(mutex_B);
    std::cout << "Thread 2 got both locks\n";
}

int main() {
    std::thread t1(safe_thread_1);
    std::thread t2(safe_thread_2);

    t1.join();  // ✅ No deadlock
    t2.join();
}
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Code Examples

📌 5. Using std::lock for Deadlock-Free Multi-Locking

`std::lock()` uses a deadlock-avoidance algorithm, making it safe regardless of argument order.

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex mutex_A;
std::mutex mutex_B;

void atomic_multi_lock_1() {
    std::lock(mutex_A, mutex_B);  // ✅ Atomic lock, no deadlock

    std::lock_guard<std::mutex> lock_A(mutex_A, std::adopt_lock);
    std::lock_guard<std::mutex> lock_B(mutex_B, std::adopt_lock);

    std::cout << "Thread 1 got both locks\n";
}

void atomic_multi_lock_2() {
    std::lock(mutex_B, mutex_A);  // ✅ Order doesn't matter with std::lock

    std::lock_guard<std::mutex> lock_B(mutex_B, std::adopt_lock);
    std::lock_guard<std::mutex> lock_A(mutex_A, std::adopt_lock);

    std::cout << "Thread 2 got both locks\n";
}

int main() {
    std::thread t1(atomic_multi_lock_1);
    std::thread t2(atomic_multi_lock_2);

    t1.join();  // ✅ No deadlock
    t2.join();
}
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Code Examples

📌 6. Bank Account Transfer with Deadlock Prevention (1/2)

Locking accounts by address order ensures consistent lock ordering regardless of function call order.

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Code Examples

📌 6. Bank Account Transfer with Deadlock Prevention (2/2)

```
#include <iostream>
#include <thread>
#include <mutex>
#include <algorithm>

class Account {
public:
    std::mutex mtx;
    int balance;

    Account(int initial) : balance(initial) {}
};

void transfer(Account& from, Account& to, int amount) {
    // ✅ Lock in address order to prevent deadlock
    Account* first = std::min(&from, &to);
    Account* second = std::max(&from, &to);

    std::lock_guard<std::mutex> lock1(first->mtx);
    std::lock_guard<std::mutex> lock2(second->mtx);

    from.balance -= amount;
    to.balance += amount;

    std::cout << "Transferred " << amount << "\n";
}

int main() {
    Account acc1(1000);
    Account acc2(500);

    std::thread t1(transfer, std::ref(acc1), std::ref(acc2), 100);
    std::thread t2(transfer, std::ref(acc2), std::ref(acc1), 50);

    t1.join();
    t2.join();

    std::cout << "Account 1: " << acc1.balance << ", Account 2: " << acc2.balance << "\n";
}
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Code Examples

📌 7. Check-Then-Act Race Condition

The critical section must cover both the check and the action to prevent races.

```
#include <iostream>
#include <thread>
#include <mutex>
#include <unordered_map>

std::mutex mtx;
std::unordered_map<int, int> cache;

void racy_get_or_create(int key) {
    // ❌ Check outside lock
    if (cache.find(key) == cache.end()) {
        // Race window here! Another thread might insert between check and act

        std::lock_guard<std::mutex> lock(mtx);
        cache[key] = key * 2;  // Might insert duplicate work
    }

    std::lock_guard<std::mutex> lock(mtx);
    std::cout << "Value: " << cache[key] << "\n";
}

void safe_get_or_create(int key) {
    std::lock_guard<std::mutex> lock(mtx);  // ✅ Lock covers both check and act

    if (cache.find(key) == cache.end()) {
        cache[key] = key * 2;
    }

    std::cout << "Value: " << cache[key] << "\n";
}

int main() {
    std::thread t1(safe_get_or_create, 10);
    std::thread t2(safe_get_or_create, 10);

    t1.join();
    t2.join();
}
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Code Examples

📌 8. Detecting Deadlock with Timeout (1/2)

Timeout-based locking detects potential deadlocks and allows recovery (backoff and retry).

```
#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>

std::timed_mutex mutex_A;
std::timed_mutex mutex_B;

void try_with_timeout_1() {
    if (mutex_A.try_lock_for(std::chrono::milliseconds(100))) {
        std::cout << "Thread 1 got A\n";

        if (mutex_B.try_lock_for(std::chrono::milliseconds(100))) {
            std::cout << "Thread 1 got both\n";
            mutex_B.unlock();
        } else {
            std::cout << "Thread 1 timeout on B (potential deadlock detected)\n";
        }

        mutex_A.unlock();
    }
}

void try_with_timeout_2() {
    if (mutex_B.try_lock_for(std::chrono::milliseconds(100))) {
        std::cout << "Thread 2 got B\n";

        if (mutex_A.try_lock_for(std::chrono::milliseconds(100))) {
            std::cout << "Thread 2 got both\n";
            mutex_A.unlock();
        } else {
            std::cout << "Thread 2 timeout on A (potential deadlock detected)\n";
        }

        mutex_B.unlock();
    }
}
// ...continues on next slide
```

Deadlocks and Race Conditions - Patterns, Detection, and Prevention

Code Examples

📌 8. Detecting Deadlock with Timeout (2/2)

[Code continues across slides]

```
// ...continued from previous slide

int main() {
    std::thread t1(try_with_timeout_1);
    std::thread t2(try_with_timeout_2);

    t1.join();
    t2.join();
}
```

What You Learned

Key Takeaways

- ✓ Overview
- ✓ Understanding Race Conditions
- ✓ The Four Coffman Conditions for Deadlock
- ✓ Deadlock Strategies

→ *Swipe to learn* →